

QTc解析アプリ (qtc.html) 仕様書

ver 2.1 : 2026/8/17 update 本仕様書はソースコード (qtc.html) の実装を正確に記述したものである。

ver 2.1 の主な変更点 Q-onset 検出アルゴリズムを全面的に改めた (§4.5、§4.6)。従来は陽性QRS／陰性QRS で 別ロジックを持ち、陰性QRS では `baseline × 0.95` の片側閾値を用いていたため、rS型波形で マーカーがR波の上やS波と基線の合流部に立ち、QT を約20～27ms 短く測っていた。ver 2.1 では「基線離脱点検出」に一本化し、さらに Q-onset 窓に限りトレース中心線で 精密化する二段構成とした。あわせて §2.3 (自己相関コム法によるキャリブレーション)、§4.4 (ヒストグラム最頻値+beat間一貫性制約による基線)、§4.7～4.9 (RR可変ゲート・ 接線の線形回帰・外れ値beat除外) を現行実装に合わせて記述を更新した。

1. アプリケーション概要

心電図 (ECG) 画像から QT 間隔を自動計測し、QTc (心拍補正 QT) を算出するブラウザベースの 単一 HTML アプリケーションである。標準12誘導心電図 (赤方眼紙上の黒波形) とホルター心電図 (カラー波形) の両方に対応する。

1.1 基本仕様

項目	値
紙送り速度	25 mm/sec (固定)
最大処理幅	2000 px (超過時は自動リサイズ)
画像入力数	最大2枚 (画像1: II誘導、画像2: V5誘導)
QTc補正式	Bazett (QTcB) および Fridericia (QTcF)
解析履歴	localStorage に最大20件保存 (ピン留め分は保持)
対応ブラウザ	モダンブラウザ (Canvas API, localStorage 使用)

1.2 QTc算出式

$$QTcB = QT / \sqrt{RR} \quad \dots \text{Bazett補正}$$
$$QTcF = QT / \sqrt[3]{RR} \quad \dots \text{Fridericia補正}$$

ここで QT は ms 単位、RR は秒単位である。

2. キャリブレーション（自動計測計算方法）

2.1 概要

キャリブレーションとは、画像上のピクセル数と実際の時間（秒）の対応関係を決定する処理である。心電図の紙送り速度は 25 mm/sec であるため、画像上で「25 mm が何ピクセルに相当するか」を求めることが目標となる。最終的に `pxPerSec`（1秒あたりのピクセル数）として表現される。

2.2 プリセットシステム

3つのキャリブレーションプリセットが用意されている。

プリセット名	デフォルト値	用途
100%	94 px/sec	標準12誘導心電図（等倍表示）
200%	190 px/sec	標準12誘導心電図（2倍表示）
ホルター	125 px/sec	ホルター心電図

- ・プリセット値は `localStorage`（キー: `qtc_cal_presets`）に JSON 形式で保存される。
- ・アプリ起動時に `loadPresets()` が呼ばれ、保存済みの値が復元される。保存値がない場合はデフォルト値が使用される。
- ・ユーザーが「再設定」ボタンを押すと、現在の画像で使用中の `pxPerSec` 値が選択したプリセットに上書き保存される（`saveCurrentAsPreset()`）。
- ・保存されたプリセット値は同一ブラウザ・同一オリジンであれば永続的に保持される。

2.3 自動キャリブレーション計測アルゴリズム (detectCalibration)

ver 2.1 では自己相関コム法を用いる。 従来の「赤スコア列プロファイルのピーク間距離の最頻値」方式は、方眼の細線・太線が混在すると最頻値が 1mm と 5mm の間で揺れて破綻した。自己相関はグリッド周期の倍数に等間隔のピーク列（コム＝櫛）を作るため、その歯の間隔から基本周期 1mm を安定に取り出せる。

Step 1: 赤格子マスクの列方向投影

```
yステップ = max(1, floor(height / 500)) ← 高解像度画像では間引き
```

赤/オレンジ/ピンク格子の判定条件:

```
(R - B) > 25 かつ (R - G) > 18 かつ R > 140
```

```
col[x] = その列で条件を満たした画素数
frac = 全格子画素数 / サンプル総数
frac < 0.003 → 検出失敗（オレンジ格子不足）
```

Step 2: 自己相関の算出

```
平均を除去: cen[x] = col[x] - mean(col)
ac0 =  $\sum \text{cen}[x]^2$ 
lim = min(260, floor(width / 2))
ac[d] = ( $\sum \text{cen}[x] \cdot \text{cen}[x+d]$ ) / ac0      (d = 0..lim)
```

Step 3: コムの歯（局所ピーク）の収集

```
d = 3..lim-1 について:
  ac[d] > ac[d-1] かつ ac[d] ≥ ac[d+1] かつ ac[d] > 0.12 → ピークとして採用
ピーク数 < 3 → 検出失敗
```

Step 4: 基本周期（1mm間隔）の精緻化

```
連続ピーク間隔 diffs の中央値 medDiff を求める
0.6 × medDiff < d < 1.4 × medDiff を満たす間隔のみ平均 → pFine (1mm の px 数)
pxPerSec = round(pFine × 25)      ← 25mm = 1秒
```

Step 5: 信頼度判定

```
以下を全て満たすとき confident = true:
  2 ≤ pFine ≤ 40
  ピーク数 ≥ 4
  60 ≤ pxPerSec ≤ 900
  ac[最初のピーク] > 0.15
confident = false → pxPerSec = 0（検出失敗）
```

2.4 検出値の採用（ver 2.1 仕様）

ver 2.1 では検出値をプリセットへ丸めない。グリッドから実測した px 値をそのまま pxPerSec として採用する（calMethod = 'auto-grid'）。プリセットは「波形抽出モード（standard / holter）の決定」にのみ使用する。

検出成功時:

```
pxPerSec = calResult.pxPerSec      ← 実測値をそのまま採用
calMethod = 'auto-grid'
nearest = findClosestPreset(pxPerSec)
extractionMode = (nearest === 'holter') ? 'holter' : 'standard'
```

画面表示: 「自動計測: 25mm(1s) = XX px (グリッド実測)」

検出失敗時:

デフォルト値 94 px/sec に最も近いプリセットを選択 (通常は 100%)

これにより 1mm = 7.5px のような非整数倍率の画像でも実測精度でQTを算出できる。

2.5 2枚目画像のキャリブレーション

画像2 (V5誘導) が読み込まれた際、画像1 (II誘導) が既に解析済みであれば、画像1と同一のキャリブレーション値 (pxPerSec, calMethod, extractionMode) がそのまま引き継がれる。画像2に対して detectCalibration() は実行されない。

2.6 手動キャリブレーション修正

キャンバス上に表示される H型バー (オレンジ色) をドラッグすることで、手動でキャリブレーション 値を調整できる。

H型バー:

左端ハンドル: 基準位置の移動 (表示位置のみ変更、pxPerSec は不変)

右端ハンドル: pxPerSec を動的に変更

```
newPxPerSec = max(20, round(マウスX座標 - calX))
```

手動修正時:

```
calMethod = 'drag' に設定
```

もう一方の画像にも同じ pxPerSec が同期される (syncCalibrationToOther)

3. 心電図波形の認識方法

3.1 抽出モードの分岐

画像から1次元信号 (各x座標における波形のy座標) を抽出する処理は、extractionMode に応じて2つのアルゴリズムに分岐する。

```
extractionMode === 'standard' → extractSignalStandard() ... 赤方眼紙上の黒波形
extractionMode === 'holter'   → extractSignalHolter()   ... カラー波形（ホルター）
```

3.2 Standard モード: 赤方眼紙上の黒波形抽出 (extractSignalStandard)

標準12誘導心電図を想定したアルゴリズムで、赤い方眼紙上に印刷された黒い波形を検出する。

Pass 1: 各列の最小輝度の算出

走査範囲: $y = 1 \sim \text{height} - 1$ (上下1pxをマージン)

各列(x)について:

赤ピクセル除外条件: $R > 100$ かつ $R > G + 30$ かつ $R > B + 30$

→ この条件に該当するピクセル（赤方眼線）はスキップ

輝度: $\text{lum} = 0.299 \times R + 0.587 \times G + 0.114 \times B$

darkestLum = その列における最小輝度

Pass 2: 最暗ピクセルの中から最も極端な位置を選択

$\text{darkestLum} < 250$ の場合（信号が存在する列）:

$\text{lumThresh} = \text{darkestLum} + 30$ (許容範囲)

$\text{centerY} = \text{height} / 2$ (画像中央)

輝度が lumThresh 以下のピクセルの中から、画像中央から最も遠い y 座標を選択

$\text{signal}[x] = \text{height} - \text{bestY}$ (y 軸を反転して上が正)

この「中央から最も遠い」選択戦略により、R波のような大きな振れ幅を持つ波形成分において、にじみや太い線による複数候補がある場合でも、波形のピーク端に近いピクセルが優先的に選択される。

注意 (ver 2.1 で明文化) この戦略には副作用がある。基線が画像中央より上にある通常の 配置では「中央から最も遠い」=上端になるため、**基線から画像中央側へ向かう下向き偏位（Q波・QS型の初期下行）はトレース太さの分だけ潰れて浅く記録される。** 実測例では 深さ14pxのq波が抽出信号上では3pxに縮み、基線のゆらぎに埋もれた。R波振幅の保持には この戦略が必要なため抽出関数は変更せず、Q-onset 検出窓に限りトレース中心線を用いる (§4.6 Part 2)。

補間処理

信号が取得できなかった列 (NaN) に対して線形補間を実施する。

NaN の列 i に対して:

左側の最も近い有効値 l と右側の最も近い有効値 r を探索

両方存在: $\text{signal}[i] = \text{signal}[l] + (\text{signal}[r] - \text{signal}[l]) \times (i - l) / (r - l)$

左のみ: $\text{signal}[i] = \text{signal}[l]$

右のみ: $\text{signal}[i] = \text{signal}[r]$

両方なし: $\text{signal}[i] = \text{height} / 2$

3.3 Holter モード: カラーチャンネル優先抽出 (extractSignalHolter)

ホルター心電図を想定したアルゴリズムで、黒・緑・赤の3チャンネルに対応する。

```
yMargin = max(2, floor(height × 0.02)) ← 画像端のアーティファクト除外
yStart = yMargin
yEnd = height - yMargin
```

黒チャンネル (channel === 'black')

```
Pass 1: minLum = min(0.299×R + 0.587×G + 0.114×B) ← 全ピクセル対象
Pass 2: threshold = min(minLum + 50, 200)
判定条件:
    lum ≤ threshold (十分に暗い)
    sat < 40 (彩度が低い = 無彩色 = 黒系) * sat = max(R,G,B) - min(R,G,B)
重み w = threshold - lum + 1 (暗いほど重い)
signal[x] = height - (Σ(y × w) / Σw) ← 重み付き重心
```

緑チャンネル (channel === 'green')

```
判定条件: G > 40 かつ G > R + 15 かつ G > B + 15
重み w = max(1, G - max(R, B)) (緑の優位度)
signal[x] = height - (Σ(y × w) / Σw)
```

赤チャンネル (channel === 'red')

```
判定条件: R > 40 かつ R > G + 15
重み w = max(1, R - G) (赤の優位度)
signal[x] = height - (Σ(y × w) / Σw)
```

補間処理 Standard モードと同一のロジックで NaN 列を線形補間する。

3.4 平滑化处理

抽出された信号に対して3点移動平均による平滑化を適用する。

```
smooth[0] = signal[0]
smooth[width-1] = signal[width-1]
smooth[i] = (signal[i-1] + signal[i] + signal[i+1]) / 3    (i = 1 ~ width-2)
```

R波の振幅保持を考慮し、意図的に窓幅を最小（3点）に設定している。

さらに、ver 2.1 の Q-onset 検出および接線探索では、単一ピクセルノイズに頑健な **平滑中央差分 (±2px)** を勾配として用いる。

```
smoothedSlope(sm, k) = (sm[k+2] - sm[k-2]) / 4
```

3.5 R波検出アルゴリズム (detectRPeaks)

Step 1: スコア信号の生成 2つの特徴量を組み合わせた複合スコアでR波候補を評価する。

```
(A) 勾配: slope[i] = |smooth[i+1] - smooth[i-1]| (中央差分の絶対値)

(B) 信号中央値 (エッジマージン 15px 除外) :
    interiorSmooth = smooth[edgeMargin .. width-edgeMargin] をソート
    signalMedian = 中央値

(C) 正規化振幅偏差:
    maxAbsDev = max(|smooth[i] - signalMedian|) (エッジ除外内部領域)
    normDev[i] = |smooth[i] - signalMedian| / maxAbsDev

(D) 正規化勾配:
    slopeMax = max(slope[i]) (エッジ除外内部領域)
    normSlope[i] = slope[i] / slopeMax

(E) 複合スコア:
    scoreSignal[i] = normDev[i] + normSlope[i] × 3.0
    → 勾配に3倍の重みを付与 (QRSの急峻な立ち上がり/下がり重視)
```

Step 2: グローバル閾値によるR波候補検出 (Pass 1)

```
globalThreshold = interiorMaxScore × 0.40 (内部領域最大スコアの40%)  
minRDist = pxPerSec × 0.45 (最小RR間隔: 450ms → HR 133bpm相当)
```

ピーク条件:

```
scoreSignal[i] > scoreSignal[i-1] (左隣より大きい)  
scoreSignal[i] > scoreSignal[i+1] (右隣より大きい)  
scoreSignal[i] > globalThreshold
```

前回ピークとの距離 > minRDist

距離 ≤ minRDist の場合はスコアが高い方を残す

Step 3: テンプレートマッチングによるギャップ補填 (Pass 2) 検出済み R-peak が2個以上の場合に実施する。

```
(A) medianRR = 全RR間隔のメディアン  
gapThresholdRR = medianRR × 1.5 (1.5倍以上はギャップと判定)  
  
(B) QRSテンプレート生成:  
halfW = max(5, floor(pxPerSec × 0.04)) ← ±40ms の窓幅  
templateLen = halfW × 2 + 1  
template = 全検出R-peak周辺の平均波形  
  
(C) tMean = テンプレートの平均値, tStd = テンプレートの標準偏差  
  
(D) ギャップ内で正規化相互相関 (NCC) を算出:  
探索範囲: 前のR-peak + minRDist ~ 次のR-peak - minRDist  
NCC = Σ((signal - sMean)(template - tMean)) / (sStd × tStd)  
NCC > 0.4 (相関閾値) かつ最大の位置をR候補として挿入
```

Step 4: QRS極性の判定

```
各R-peak位置の信号値と signalMedian を比較:  
signalMedian より上: aboveCount++  
signalMedian より下: belowCount++  
  
belowCount > aboveCount → isInvertedQRS = true
```

注意 (ver 2.1 で明文化) この判定は「最大偏位がどちら向きか」を見ているだけであり、QRSの形そのものを分類していない。**rS型 (小さいr波+深いS波)** では **S波の谷が rLoc** として 検出され、**isInvertedQRS = true** となる。このとき真の QRS 開始点は rLoc より r波の分だけ手前にあ

り、r波が「基線への復帰」の壁として立ち上がる。§4.6 の 連続性条件はこの壁を越えるために必要である。

Step 5: ローカル極値へのスナップ

陽性QRS: 各R-peak 周辺 $\pm 10\text{px}$ で信号最大値の位置に移動

陰性QRS: 各R-peak 周辺 $\pm 20\text{px}$ で信号最小値の位置に移動

Step 6: RR間隔整合性フィルタ

メディアンRR間隔の65%未満の間隔が存在する場合:

その間隔を構成する2つのピークのうち、scoreSignal が低い方を除去

これを短すぎる間隔がなくなるまで繰り返す (ただし R-peak が3個未満になったら停止)

4. 接線法によるQT間隔計測

4.1 概要

QT間隔は Q-onset (Q波の開始点) から T-end (T波の終了点) までの時間であり、本アプリでは接線法 (tangent method) により T-end を決定する。接線法は、T波の下行脚 (陽性T波の場合) または上行脚 (陰性T波の場合) における最急勾配点に接線を引き、その接線と基線の交点を T-end とする方法である。

4.2 ビート解析の処理フロー (analyzeBeats)

ver 2.1 では、Q-onset の精密化方式を画像単位で決めるため、処理を3つのパスに分けている。

Pass 1 : 各beatの粗Q-onset推定と基線候補の算出

↓

基線一貫性: 全beatの基線候補から globalBase と許容偏差 tol を決定

↓

Pass 2a : 各beatの基線を確定 (手動 > 一貫性制約 > 候補)

Pass 2b : Q-onset検出

① coarseQ0nset() ... 基線離脱点の粗検出 (全beat)

② refineQ0nset() ... 中心線による精密化の可否判定と実行 (全beat)

③ 画像単位の多数決で採否を決定 → state.q0nsetMode

Pass 2c : T波極性判定 → T-peak検出 → 接線法によるT-end決定 → QT/QTc算出

↓

外れ値beat除外: ストリップ内のQT中央値から大きく外れるbeatを除外

4.3 Pass 1: 粗Q-onset推定

roughQ は基線推定区間の右端を決めるためだけに使用される。ver 2.1 では探索窓を精密検出と同じ 120ms に揃え、サニティガードを追加した。

探索範囲: $qLimit = \max(prevR + 10, \text{floor}(rLoc - 0.12 \times pxPerSec)) \leftarrow 120ms$

陽性QRS: R-peak位置から左へ1pxずつ辿る

$smooth[k] \leq smooth[roughQ] \rightarrow roughQ = k$ (より低い点を更新)

$smooth[k] > smooth[roughQ] + 3 \rightarrow$ 探索終了

陰性QRS: R-peak (谷 = nadir) から左へ辿る

$smooth[k] \geq maxVal \rightarrow maxVal$ 更新, $roughQ = k$

$smooth[k] < maxVal - 3 \rightarrow$ 探索終了

サニティガード (ver 2.1 追加) :

$roughGuard = \max(3, \text{floor}(0.01 \times pxPerSec))$

$roughQ > rLoc - roughGuard$ の場合 $\rightarrow roughQ = \max(qLimit, rLoc - roughGuard)$

ガードがない場合、R頂点スナップが真の極値からわずかにずれていると初回の比較で即座に探索が終了し、**roughQ** が R頂点に張り付く。

4.4 Pass 2a: 基線 (Baseline) 算出

ヒストグラム最頻値方式 (estimateIsoelectric)

前R波後〜Q直前の区間では、等電位部 (TP/PRセグメント) が最も長く一定レベルに留まる。したがって輝度(y)ヒストグラムの最頻ビンが等電位レベルに一致する。T波 (上下に掃引) や ST偏位に頑健で、旧「最小分散窓」方式が平坦なT波頂点/P波を誤検出する問題を解消する。

探索区間: $start = \max(prevR + \text{floor}(0.10 \times pxPerSec), prevR + 6)$

$end = \max(roughQ - 3, start + 1)$

区間が4px未満の場合: Q直前15pxの中央値を返す

ビン幅 2px でヒストグラムを作成 → 最頻ビン bestBin
最頻ビン ± 1 bin 内のサンプルの中央値を等電位レベルとする

beat間一貫性制約（ドリフト抑制）

全beatの等電位候補の中央値を基準 `globalBase` とし、許容偏差 `tol`（MADベース）を超える 外れ値のみ基準近傍へ引き戻す。真の緩やかな基線変動は許容しつつ、誤検出による上下ブレを 除去する。

```
globalBase = 全beatの baseCand の中央値
mad = |baseCand - globalBase| の中央値
tol = max(6, 3 × mad)

各beatについて:
    baseCand が無効          → globalBase を採用
    |baseCand - globalBase| > tol → globalBase 近傍で最も平坦な窓を再探索
                                   (estimateIsoelectricNear)
                                   見つからなければ globalBase にクランプ
    それ以外                  → baseCand をそのまま採用
```

手動基線 ユーザーが基線（青い水平線）を上下にドラッグした場合、`manualBaselines[beatId]` に保存され、自動算出を上書きする。

4.5 Pass 2b Part 1: 基線離脱点検出 (coarseQOnset) — ver 2.1 新方式

設計方針 Q-onset の定義は「トレースが基線を離れてQRSに移行する点」である。想定される3形態は次のいずれかで、すべて同一の判定で扱える。

1. Q波がある場合 … 基線からQ波へ移行する点
2. Q波がなくそのままR波になる場合 … 基線からR波へ移行する点
3. QS pattern … 基線から下向きに下がり始める点

アルゴリズム 「R極値から後方へ遡り、最初に『基線上かつ平坦』が N px 連続する区間を 見つけ、その区間のR側の端を onset とする」。判定に `|smooth - baseline|` を用いるため、**従来の陽性QRS／陰性QRS の分岐は不要になり1本化された。**

```
pxPerMm = pxPerSec / 25
qLimit  = max(prevR + 10, floor(rLoc - 0.12 × pxPerSec))    ← 120ms窓
A        = |smooth[rLoc] - baseline|                        ← QRS振幅
ampTol   = max(4, min(0.50×pxPerMm, max(0.15×pxPerMm, 0.06×A)))
maxSlope = max |smoothedSlope(k)|    (k = qLimit..rLoc)
```

```

slopeTol = max(0.02×pxPerMm, 0.10 × maxSlope)
N          = max(3, round(0.010 × pxPerSec))          ← 持続10ms

k = rLoc-1 → qLimit と後方走査:
|smooth[k] - baseline| ≤ ampTol かつ |smoothedSlope(k)| ≤ slopeTol
→ run++, run ≥ N になったら coarse = k + run - 1 (runのR側の端)
条件を満たさない → run = 0 (リセット)

```

連続N条件が要点である。 Q波の谷底も「基線近傍かつ勾配 ≈ 0 」を満たしうるが、谷では 1～数px しか条件が続かないため run が成立しない。走査は谷とr波を通過して真のPRセグメントに 到達する。これにより形態1～3が同一ロジックで正しく扱える。

閾値をpx/mmとQRS振幅にスケールさせる理由 旧実装の固定px閾値 (+3、baseline × 0.95) は 画像解像度に依存して意味が変わった。とくに baseline × 0.95 は基線の絶対px座標に比例するため、基線が y=677px にあれば基線から34px下、y=60px なら3px下という隠れパラメータになっていた。

ampTol の絶対下限 4px 「0.5mm相当、ただし4px未満にはしない」。1mm < 8px の画像ではPRセグメントの緩い下降（基線推定はTP部に引かれるため数px低く出る）が 0.5mm を超え、run が途切れて停止位置がbeat毎に数pxずれる。

フォールバックとサニティガード

```

run が成立しない場合:
① 平坦条件のみで判定: |smoothedSlope(k)| ≤ 0.20 × maxSlope の最初の点
② それも無ければ qLimit

サニティガード:
guard = max(3, floor(0.01 × pxPerSec))
coarse > rLoc - guard の場合 → coarse = rLoc - guard

```

このガードにより「マーカがR頂点に立つ」症状を構造的に排除する。

4.6 Pass 2b Part 2: 中心線による精密化 (extractCenterline / refineQOnset)

必要性 §3.2 の注意のとおり、extractSignalStandard は「画像中心から最も遠い画素」を採るため、基線から画像中心側へ向かう下向き偏位（Q波・QS初期下行）がトレース太さの分だけ 潰れる。Q波そのものが抽出信号上に存在しないため、Part 1 だけでは「基線からQ波に移行する 部分」を検出できない。そこで Q-onset 窓に限り、生画素からトレース重心（中心線）を 再抽出して離脱点を確定する。

中心線抽出 (extractCenterline)

```
CL_LUM_THRESHOLD = 160
```

各列 x について:

赤方眼を除外 ($R > 100$ かつ $R > G + 30$ かつ $R > B + 30$) した上で

$lum < CL_LUM_THRESHOLD$ の画素を連続run単位にまとめ、各runの輝度重心を求める

重み: $w = CL_LUM_THRESHOLD - lum$

前列の値に最も近いrun (重心・上端・下端のいずれかの距離が最小) を採用 ← トレース追従

暗画素が無い列は前列値を維持

離脱点の確定 (refineQOnset)

```
back      = max(N, round(0.020 × pxPerSec))    ← 精密化する区間 20ms
fwd       = max(1, round(0.004 × pxPerSec))
noiseLen  = max(10, floor(0.040 × pxPerSec))    ← 基線ノイズ計測窓 40ms
bandStart = coarse - back
nStart    = max(prevR + 5, bandStart - noiseLen)
```

μ = 中心線[nStart .. bandStart] の中央値

$\maxDev = \max | \text{中心線}[nStart .. bandStart] - \mu |$

【解像度ゲート】

$\text{ceiling} = 0.40 \times \text{pxPerMm}$ ← 生理的離脱幅の上限 $\approx 0.4\text{mm} (0.04\text{mV})$

$\text{tauRaw} = 1.2 \times \maxDev$

$\text{tauRaw} > \text{ceiling} \rightarrow$ 基線ノイズ過大。精密化を行わず coarse を返す ($\text{gateOK} = \text{false}$)

$\text{tau} = \max(\text{tauRaw}, 0.10 \times \text{pxPerMm})$

$M = \max(2, \text{round}(0.004 \times \text{pxPerSec}))$

band = [bandStart, coarse + fwd] を前方走査:

$| \text{中心線}[x] - \mu | > \text{tau}$ が M px 持続する最初の $x \rightarrow \text{onset}$

見つからなければ coarse

設計上の要点 (いずれも実測で確認した失敗を回避するためのもの)

- τ は最大偏位から決める。MAD等のロバスト推定を使うと基線のゆらぎを外れ値として捨ててしまうため τ が小さくなりすぎ、ゆらぎで誤発火して6px早い側に外れた。
- 走査方向は前方でなければならない。後方走査 (基線帯への再入を探す) にすると q波からr波へ移る際の基線交差点で誤停止する。band内の極値を基準にすると、q波より大きいr波に極値を取られる。

- ・解像度ゲートにより低解像度画像では自動的に Part 1 の結果になる。これがないと 1mm ≒ 5.7px の画像で退行した。

Part 2 の採否は画像単位で決める（多数決）

```
gateOK となったbeat数 × 2 > 全beat数 → useRefine = true
state.qOnsetMode = useRefine ? 'centerline' : 'coarse'
各beatの Q-onset = (useRefine かつ そのbeatが gateOK) ? refinedQ : coarseQ
```

beat毎にゲート判定すると同一画像内で方式が混在し、beat間で系統的に1~2pxずれてQTのばらつきが増えるため、画像単位で揃える。

手動 Q-onset `manualQOnsets[beatId]` が存在する場合は常にそれを優先する。

4.7 Pass 2c: T波の探索ゲートと極性判定

高心拍（≈150bpm, RR≈400ms）ではT波・P波が接近するため、固定時間ゲートでは探索窓が潰れる、またはT頂点を取りこぼす。各ゲートを「絶対時間」と「RRの割合」の小さい方に制限し、低心拍では従来同等、高心拍ではRR比で自動的に圧縮して破綻を防ぐ。

```
rrLen = nextR - rLoc

tStart = floor(rLoc + min(max(0.10×pxPerSec, 16), 0.18 × rrLen))
    ← QRS直後のJ点相当
pGuard = min(max(0.12×pxPerSec, 18), 0.28 × rrLen)
    ← 次R手前のP波ガード
tLimit = min(nextR - pGuard,
    rLoc + min(max(0.80×pxPerSec, 0.55×rrLen), 0.85×rrLen))
    ← T波下行脚の探索上限
tLimit ≤ tStart → そのbeatは棄却

tPeakLimit = max(tStart + 1,
    min(tLimit, nextR - min(max(0.14×pxPerSec, 24), 0.30×rrLen)))
    ← T-peak探索はさらに手前で打ち切りP波の誤検出を防ぐ
```

T波極性の自動判定

```
tStart ~ tPeakLimit 内の最大値 tMaxVal と最小値 tMinVal を取得
|tMinVal - baseline| > |tMaxVal - baseline| → 陰性T波 (isNegativeT = true)
それ以外 → 陽性T波
```

T-peak検出

陽性T波: $tStart \sim tPeakLimit$ 内で smooth の最大値の位置

陰性T波: $tStart \sim tPeakLimit$ 内で smooth の最小値の位置

T-peak振幅フィルタ: $|smooth[tPeak] - baseline| < 3 \rightarrow$ ノイズとして棄却

4.8 Pass 2c: 接線法によるT-end決定

これが本アプリの核心となるアルゴリズムである。

Step 1: 下行脚探索範囲の決定

```
slopeEnd = min(tLimit, width - 1)
```

基線復帰点による制限:

T-peak の次から、陽性Tなら $smooth[k] \leq baseline$ 、陰性Tなら $smooth[k] \geq baseline$

となる最初の点を descEnd とする

→ T波の後に出現する心房ペーシングスパイク/P波の急峻な勾配の誤検出を防ぐ

T-P谷による追加制限（高心拍対策）:

$minGap = \max(2, \text{round}(0.02 \times pxPerSec))$ ← T頂点直後の微小ゆらぎを無視

$turnTh = 0.03$ ← 反転とみなす平滑勾配の閾値

陽性T: まず下行($s < -turnTh$)を確認し、その後上行($s > turnTh$)へ転じる谷で終了

陰性T: まず上行($s > turnTh$)を確認し、その後下行($s < -turnTh$)へ転じる山で終了

$descEnd = \min(descEnd, nadir)$

→ T波がP波と融合し基線へ戻らない場合でも終末下行脚だけを対象にできる

Step 2: 最急勾配点の探索

平滑中央差分 $smoothedSlope(\pm 2px)$ を用いる（単一ピクセルノイズに頑健）

陽性T波: $tPeak \sim descEnd$ で s が最も負の位置を slopeIdx とする

陰性T波: $tPeak \sim descEnd$ で s が最も正の位置を slopeIdx とする

slopeValid = 陽性なら $bestSlope < 0$ 、陰性なら $bestSlope > 0$

Step 3: 線形回帰による接線の安定化

$reg = \max(2, \text{round}(0.012 \times pxPerSec))$ ← 最急点近傍±約12ms

slopeIdx ± reg の区間で最小二乗回帰 → regSlope, intercept

回帰勾配が正しい符号のときのみ採用（誤った平坦化を防ぐ）

```

contactX = slopeIdx
contactY = regSlope × slopeIdx + intercept
tEnd = (baseline - intercept) / regSlope    ← 接線と基線の交点

```

Step 4: フォールバック処理

勾配が有効でない場合:

フォールバック 1: T-peak の2px後から、信号が基線の $\pm 2px$ 以内に返る最初の点
 フォールバック 2: T-peak の2px後から、基線との距離が最も小さい点
 いずれも得られなければ そのビートは棄却

Step 5: T-endの妥当性

$!isFinite(tEnd)$ または $tEnd \leq contactX$ または $tEnd > slopeEnd \rightarrow$ そのビートは棄却
 (画像端で切れたT波・平坦すぎるT波を捏造せず除外する)

4.9 Pass 2c: QT/QTc算出と外れ値beatの除外

```

QT (ms) = (tEnd - qOnset) / pxPerSec × 1000
RR (sec) = (rLoc - prevR) / pxPerSec
HR (bpm) = 60 / RR
QTcB = QT / √RR
QTcF = QT / √RR

```

生理学的範囲チェック: $200 \leq QT \leq 700$ ms の範囲外のビートは棄却

外れ値beatの除外 (平均QT/QTcの汚染防止)

ストリップ内のQT中央値から大きく外れるbeatを頑健に除外する。対象は画像端で切れたT波による過小/過大QT、R波検出漏れによる二重RR間隔beatである。手動修正済みbeatは常に保持する。

beats が3個以上の場合:

```

medQT = 全beatのQTの中央値
medRR = 全beatのRRの中央値
qtTol = max(60, 0.22 × medQT)

```

残す条件: `b.isManual` または

$(|b.qt - medQT| \leq qtTol \text{ かつ } medRR \times 0.5 \leq b.rr \leq medRR \times 1.6)$

残ったbeatが2個以上のときのみ置き換える

4.10 手動修正による接線の再計算

ユーザーが赤マーカー（接線の接触点）をドラッグした場合、接触点の x 座標が `manualTEnds[beatId]` に保存される。

手動接触点 cx での処理:

```
cSlope = (smooth[cx+1] - smooth[cx-1]) / 2  ← 中央差分で局所勾配を算出
cy = smooth[cx]
tEnd = (baseline - cy) / cSlope + cx        ← 接線と基線の交点
(cSlope === 0 の場合: tEnd = cx + 20 にフォールバック)
```

4.11 統計量の算出

全ビート（外れ値除外後）の計測値の算術平均が最終結果として表示される。

```
HR    = mean(全ビートの hr)
QT    = mean(全ビートの qt)
QTcB  = mean(全ビートの qtC_B)
QTcF  = mean(全ビートの qtC_F)
```

5. ホルター心電図の判定方法

5.1 ホルターモードへの切り替え条件

条件 1: 自動キャリブレーション検出値が、ホルタープリセットに最も近い場合

- `findClosestPreset()` がホルターを返す
- `extractionMode = 'holter'` が自動設定

条件 2: ユーザーが手動で「ホルター」プリセットボタンを選択した場合

- `selectPreset('holter')` が呼ばれる
- `extractionMode = 'holter'` が設定される

5.2 ホルターモードと標準モードの違い

処理	標準モード (standard)	ホルターモード (holter)
波形抽出関数	<code>extractSignalStandard()</code>	<code>extractSignalHolter()</code>
赤方眼フィルタ	あり（赤ピクセルを除外）	なし

処理	標準モード (standard)	ホルターモード (holter)
波形色	黒のみ	黒/緑/赤 (チャンネル選択)
信号取得方式	最暗ピクセル追跡 (中央から最遠)	重み付き重心
チャンネル選択UI	非表示	表示
画像端マージン	上下各1px	上下各2% (最低2px)
Q-onset 中心線精密化	適用対象	抽出信号が既に重心のため通常はゲートで非適用

5.3 最適チャンネル自動検出 (detectBestChannel)

画像読み込み時にホルターモードが選択された場合、最適な波形チャンネルが自動判定される。

サンプリング間隔: $\text{step} = \max(1, \text{floor}(\text{width} / 200))$

→ 画像全体から約200列をサンプリング (計算量削減)

黒スコア (blackScore): $\text{lum} < 80$ かつ $\text{sat} < 40$ → 暗くて彩度が低いピクセル

緑スコア (greenScore): $G > 40$ かつ $G > R + 15$ かつ $G > B + 15$

赤スコア (redScore) : $R > 40$ かつ $R > G + 15$

選択ロジック:

$\text{colorThreshold} = \text{blackScore} \times 0.3$

- (1) 緑が赤より多く、かつ colorThreshold を超え、
かつ blackScore の50%以上 → 'green' を選択
- (2) 赤が緑より多く、かつ colorThreshold を超え、
かつ blackScore の50%以上 → 'red' を選択
- (3) それ以外 → 'black' を選択 (デフォルト)

この選択ロジックにより、明確なカラー波形がある場合のみカラーチャンネルが選択され、カラー成分が少ない場合は安全に黒チャンネルにフォールバックする。

5.4 手動チャンネル切り替え

ユーザーがチャンネルバー上の「黒」「緑」「赤」ボタンをクリックすると、`switchChannel()` が呼ばれ、波形抽出からやり直される。

```
switchChannel(index, channel):
    state.manualChannelSelected = true    ← 以降の自動検出を抑制
    state.channel = channel
    performSignalExtraction(state)        ← 波形抽出→R波検出→ビート解析を再実行
```

`manualChannelSelected = true` が設定されると、`performSignalExtraction()` 内で `detectBestChannel()` がスキップされ、ユーザーが選択したチャンネルが維持される。

5.5 ホルターモードの波形抽出の特徴

標準モードとの最大の違いは、信号の取得方式にある。

標準モード：

- 「最暗ピクセル → 画像中央から最も遠いピクセル」を選択
- 1px の離散的な位置が返される
- R波のような大きな振れ幅に対して先端側を優先

ホルターモード：

- 「条件を満たす全ピクセルの重み付き重心」を算出
- サブピクセル精度の連続値が返される
- 波形の幅の中心を追跡するため、安定した信号が得られる
- 太い波形線でも正確な追跡が可能

ver 2.1 の Q-onset 中心線精密化 (§4.6) は、標準モードにこのホルターモードと同じ「重心」の利点を、Q-onset 窓に限って持ち込むものである。

6. ユーザーインタラクション

6.1 マーカー操作

操作	対象	動作
クリック	R-peak (青丸)	そのR-peakを削除
ドラッグ	R-peak (青丸)	R-peak位置を水平移動
Shift+クリック	キャンバス任意位置	その位置にR-peakを追加
ドラッグ	赤マーカー (接線接触点)	接線の接触点を移動 → 接線とT-endを再計算
クリック	赤マーカー	手動修正をリセット (自動検出に戻す)

操作	対象	動作
ドラッグ	金色マーカー (T-peak)	T-peakを移動 → 接線を再計算
ドラッグ	緑マーカー (Q-onset)	Q-onset位置を移動
クリック	緑マーカー	Q-onsetの手動修正をリセット
ドラッグ	青い水平線 (基線)	基線を上下に移動
クリック	青い水平線	基線の手動修正をリセット
ドラッグ	H型バー右端	キャリブレーション値を変更
ドラッグ	H型バー左端	キャリブレーション表示位置を移動

6.2 データ永続化 (localStorage)

キー	内容	形式
qtc_cal_presets	キャリブレーションプリセット値	<code>{"100": 94, "200": 190, "holter": 125}</code>
qtc_analysis_history	解析履歴 (最大20件、ピン留め分は保持)	JSON配列

7. 処理パイプライン全体図

画像読み込み (handleFile)

↓

リサイズ判定 (幅 > 2000px なら縮小)

↓

runAnalysis()

- └─ detectCalibration() ← 自己相関コム法で 1mm を実測 → pxPerSec
- └─ pxPerSec = 実測値をそのまま採用 (calMethod = 'auto-grid')
- └─ findClosestPreset() ← 抽出モードの決定にのみ使用
- └─ extractionMode 決定 ← 'standard' or 'holter'
- └─ performSignalExtraction()
 - └─ [holter] detectBestChannel() ← 最適チャンネル自動判定
 - └─ [holter] extractSignalHolter() ← カラーチャンネル重み付き重心
 - └─ [standard] extractSignalStandard() ← 最暗ピクセル追跡
 - └─ 3点平滑化
 - └─ detectRPeaks()

8.2 運用解像度での検証

実運用ファイル（783×335、1mm = 7.56px、pxPerSec 189、**1px = 5.29ms**）で、生画素から真値（トレース下端／重心が基線から離れ始める列）を確定して誤差を実測した。

beat	真値	旧実装	Part 1 のみ	Part 1+2 (ver 2.1)
1	x=172	+26.5 ms	+10.6 ms	+5.3 ms
2	x=299	+26.5	+10.6	0.0
3	x=427	+21.2	+5.3	+10.6
4	x=553	+21.2	+10.6	+5.3
5	x=680	+21.2	+10.6	0.0
平均誤差		+23.3 ± 2.6	+9.5 ± 2.1	+4.2 ± 4.0

正の誤差は「R側に遅れる = QTを短く測る」方向である。ver 2.1 は全beatで真値の ±1px 以内に入り、平均誤差 +4.2ms は 1px = 5.29ms の量子化限界そのものである。

QT / QTc への影響 旧実装はこの心電図で QT を 19.0ms、QTcB を 23.2ms 短く算出していた（RR 0.672s）。真値基準では旧実装の QTcB 過小評価は約28msである。

8.3 解像度スweep

同一心電図を各幅にリサイズして R-Q（ms）を測定した。真値は 50～52ms。

幅	1mm	旧実装	Part 1	Part 1+2
600 px	5.8 px	27.4	41.1	45.2
783 px（運用）	7.6 px	23.3	37.0	42.3
1000 px	9.7 px	23.9	38.7	46.1
1500 px	14.6 px	22.5	40.7	48.9
2000 px	19.3 px	22.8	42.2	48.9

600px まで下げても校正・R波検出とも破綻しない。解像度を上げるほど真値に漸近する。運用解像度で実用十分だが、精度を追う場合は 1500px 以上で取り込むと約5ms改善する。

8.4 既存サンプルでの回帰確認

画像	1mm	旧 R-Q (ms)	新 R-Q (ms)	qOnsetMode
運用ECG	7.6 px	23.3 ± 2.6	42.3 ± 5.8	centerline
test_QS_pattern	7.6 px	31.7 ± 0.0	47.6 ± 3.4	centerline

画像	1mm	旧 R-Q (ms)	新 R-Q (ms)	qOnsetMode
test_negativeT	7.6 px	26.4 ± 10.5	58.2 ± 13.5	centerline
test_200	7.6 px	40.8 ± 2.3	31.6 ± 3.7	coarse
test_lqt	5.7 px	26.3 ± 3.0	40.3 ± 5.8	coarse
test_high_rate_200	7.6 px	10.5 ± 0.0	41.4 ± 1.8	coarse
test_holter	6.0 px	22.9 ± 3.3	30.5 ± 3.3	coarse
test_v5	3.8 px	21.1 ± 10.6	63.2 ± 7.5	coarse
test_2	3.8 px	26.4 ± 5.3	44.7 ± 4.5	coarse

- **test_high_rate_200** の旧実装 10.5ms は R頂点の2px手前であり、「R頂点に立つ」症状が 低解像度サンプルでも再現していたことを示す。
- **test_200** で新実装のR-Qが小さくなったのは退行ではない。旧実装は40ms窓内の最小値が 平坦なPRセグメントの途中に落ちていた（平坦区間内のどこに立つかは不定）。新実装は PRセグメントの終端＝上行開始の直前を返しており、こちらが正しい。
- **test_negativeT** はbeat間ばらつきが大きいですが、旧実装でも SD 10.5ms であり本改修による 退行ではない。

8.5 運用上の注意

Part 2（中心線精密化）は解像度ではなく**基線ノイズ**でゲートされる。基線の最大偏位が 0.4mm相当を超える画像では自動的に Part 1 の結果になる。運用ECGは1mm = 7.56pxでも トレースが細く（太さ2px）基線が平坦なためゲートを通る。適用結果は画面の `state.qOnsetMode` で確認できる。

本仕様書は qtc.html ver 2.1 (2026/8/17) のソースコードに基づいて作成された。